

SISTEMAS DISTRIBUIDOS I
(TA050) CURSO PABLO D. ROCA

Trabajo Práctico Diseño

Coffee Shop Analysis

16 de septiembre de 2025

Sebastian Mauricio Vintoñuke	106063
Marcelo Agustin Origoni	109903
Agustin Pelliciari	108172

1. Alcance

El sistema tiene como propósito procesar y analizar información de ventas en una cadena de cafés en Malasia.

A través de un diseño distribuido, busca obtener métricas a partir de datos transaccionales, de clientes, tiendas y productos.

Alcance funcional, permite obtener:

- Transacciones realizadas entre 2024 y 2025 entre las 06:00 AM y las 11:00 PM con monto total mayor a 15.
- Nombre del producto más vendido y nombre del producto que más ganancias ha generado, para cada mes en 2024 y 2025.
- TPV (Total Payment Value) por cada semestre en 2024 y 2025, para cada sucursal, para transacciones realizadas entre las 06:00 AM y las 11:00 PM.
- Fecha de cumpleaños de los 3 clientes que han hecho más compras entre 2024 y 2025, para cada sucursal.

Alcance no funcional:

- Optimizado para entornos multicomputadoras.
- Soporta el incremento de los elementos de cómputo para escalar los volúmenes de información a procesar.
- Usa un Middleware para abstraer la comunicación basada en grupos.
- Soportar una única ejecución del procesamiento y provee graceful quit frente a señales SIG-TERM.

2. Arquitectura de Software

El sistema se basa en una arquitectura mixta. Adopta un modelo cliente-servidor para gestionar la interacción con los usuarios. Internamente, está conformado por múltiples componentes de cómputo que trabajan en conjunto, lo que permite atender los requerimientos de escalabilidad y aprovechar las ventajas de los entornos multicómputo.

2.1. Interacción con el cliente

La comunicación entre el cliente y el sistema se basa en un modelo cliente-servidor. El cliente envía una solicitud en la que incluye un `operationId`, que identifica la consulta a ejecutar, y los argumentos, que corresponden a uno o varios archivos CSV a procesar. El servidor recibe esta petición y responde con un `requestId`, un identificador único asociado a la consulta.

Más adelante, el cliente utiliza ese `requestId` para consultar el estado de su solicitud. Esta operación es bloqueante, ya que el cliente debe esperar hasta que el procesamiento haya finalizado para poder acceder a los resultados.

En conjunto, la interacción define un **protocolo de comunicación asíncrono**, estructurado en dos intercambios de tipo request-reply sincrónicos: el primero para registrar la consulta y obtener el identificador, y el segundo para recuperar los resultados una vez estén disponibles.

2.2. Interacción entre nodos

La interacción principal del sistema se establece entre el Scheduler y los nodos de ejecución, responsables de llevar a cabo las distintas etapas de procesamiento que conforman los pipelines.

Esta coordinación se realiza a través de un **Message-Oriented Middleware (MOM)**, centralizado y basado en un modelo de colas asíncronas de mensajes. En este esquema, cada cola representa una operación específica a ejecutar, como select, project, sort, group by o join.

3. Vista Lógica

La resolución de consultas se lleva a cabo principalmente mediante la interacción coordinada de los siguientes componentes:

Planner: Recibe el identificador de la consulta solicitada por el cliente junto con los archivos asociados. Su función es generar un plan de ejecución acorde con la consulta y optimizado para el procesamiento paralelo cuando sea posible. El plan consiste en una secuencia ordenada de tareas, donde cada tarea define el origen de los datos, la transformación a aplicar y el destino de los resultados.

Scheduler: Se encarga de recibir los planes y despachar las tareas de forma ordenada y coherente. Además, actualiza el estado de los planes incorporando los resultados parciales de las tareas ya ejecutadas.

Worker Node: Ejecuta tareas individuales pertenecientes a un plan. Para ello, accede a los datos de origen, aplica la transformación especificada y devuelve los resultados transformados. Una vez completada la tarea, notifica al scheduler para que actualice el plan correspondiente.

Middleware: Gestiona la comunicación entre el scheduler y los nodos de ejecución. Garantiza que el scheduler pueda interactuar de manera transparente y equilibrada con múltiples nodos, y que, a su vez, los nodos puedan informar al scheduler sobre la finalización de sus tareas.

Intermediate Results: Almacenamiento persistente que conserva los resultados intermedios del procesamiento. Este componente aporta resiliencia ante fallos, permitiendo reanudar o recomponer la ejecución en caso de interrupciones.

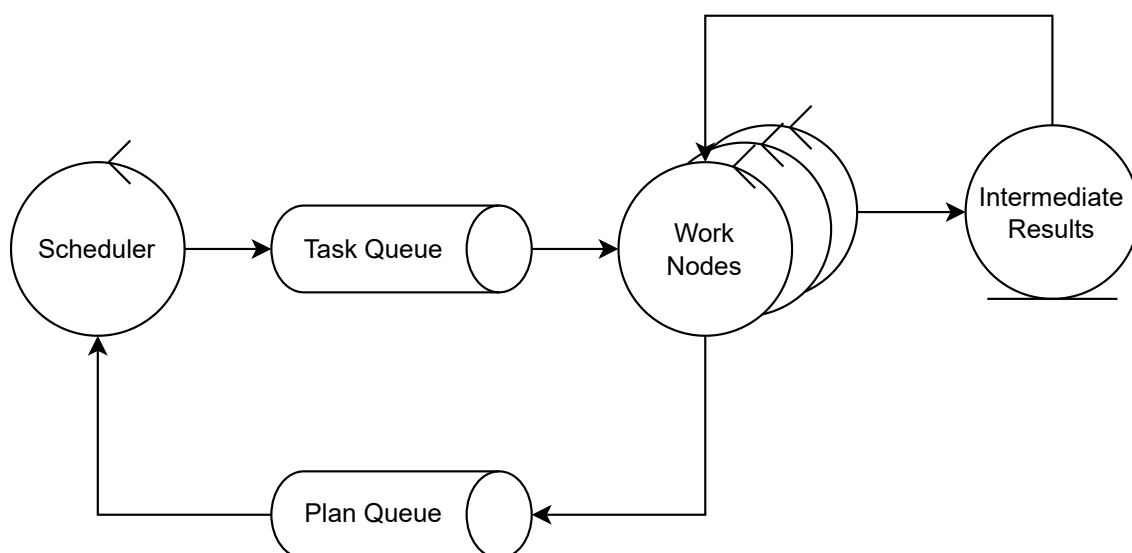


Figura 1: Diagrama de robustez entre las entidades principales

4. Vista de Procesos

La resolución de consultas se modeló siguiendo los esquemas del álgebra relacional, ampliamente utilizado en motores como SQL. Esta decisión permite aprovechar un lenguaje de dominio ya conocido y, al mismo tiempo, disponer de un modelo extensible que facilite la incorporación de nuevos tipos de consultas.

La planificación de las consultas se diseñó con el objetivo de reducir el volumen de datos lo antes posible. Para ello, se aplican en primer lugar las proyecciones, eliminando las columnas irrelevantes, y los selects, descartando las filas que no cumplen los criterios de filtrado. Este enfoque no solo optimiza la ejecución al minimizar la cantidad de datos procesados en las etapas posteriores, sino que además reduce la complejidad de operaciones más costosas como sort, group by y join.

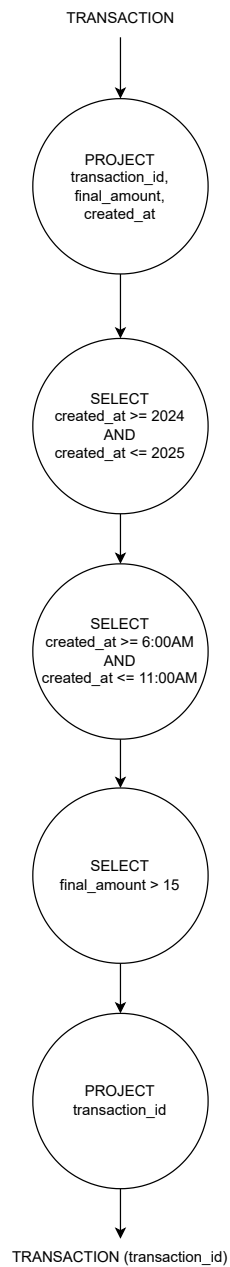


Figura 2: Diagrama DAG consulta 1°

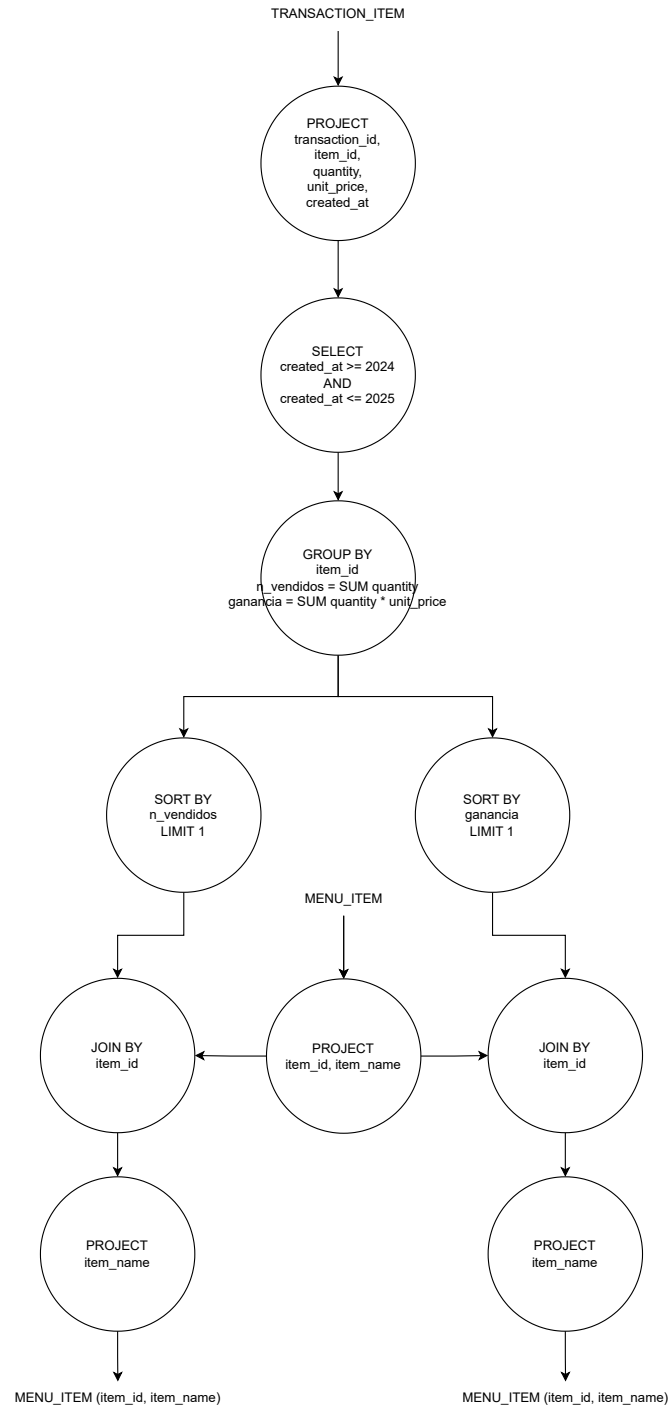


Figura 3: Diagrama DAG consulta 2°

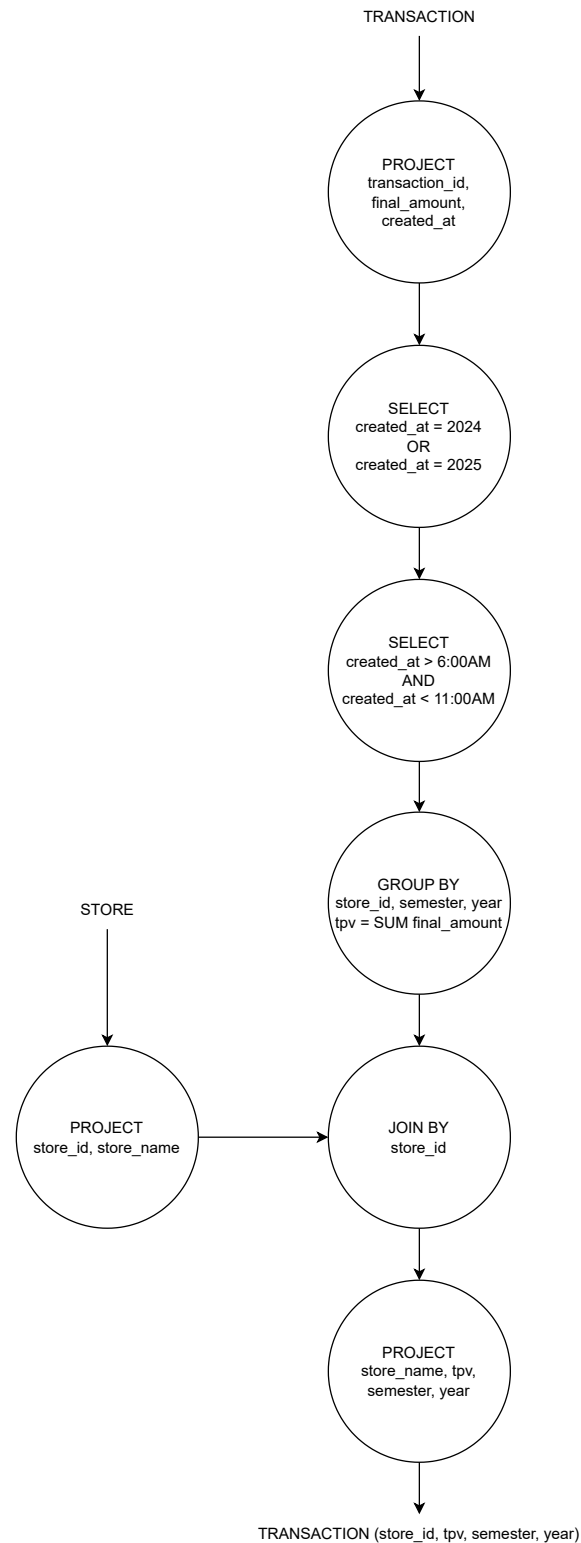


Figura 4: Diagrama DAG consulta 3°

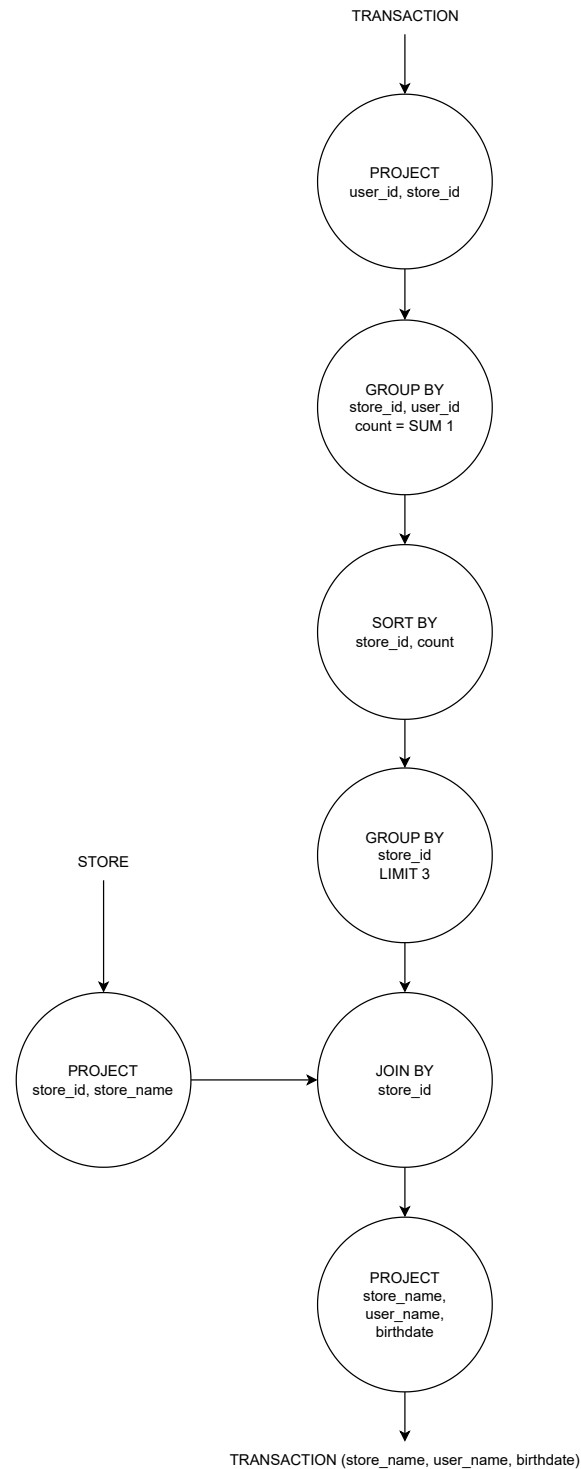


Figura 5: Diagrama DAG consulta 4°

Ciertas consultas permiten la paralelización de tareas, lo que mejora significativamente el desempeño. En términos generales, tanto las proyecciones como los filtros pueden ejecutarse en paralelo sin inconvenientes. Además, la propia estructura de algunas consultas habilita la paralelización de etapas adicionales.

Un ejemplo concreto se encuentra en la consulta número 2, en la que se requieren dos resultados distintos: uno ordenado por ganancia y otro por unidades vendidas. En este caso, es posible paralelizar la ejecución de ambos sorts, lo que permite aprovechar mejor los recursos de cómputo y reducir el tiempo total de procesamiento.

A continuación se presenta un diagrama de actividades que ilustra esta situación:

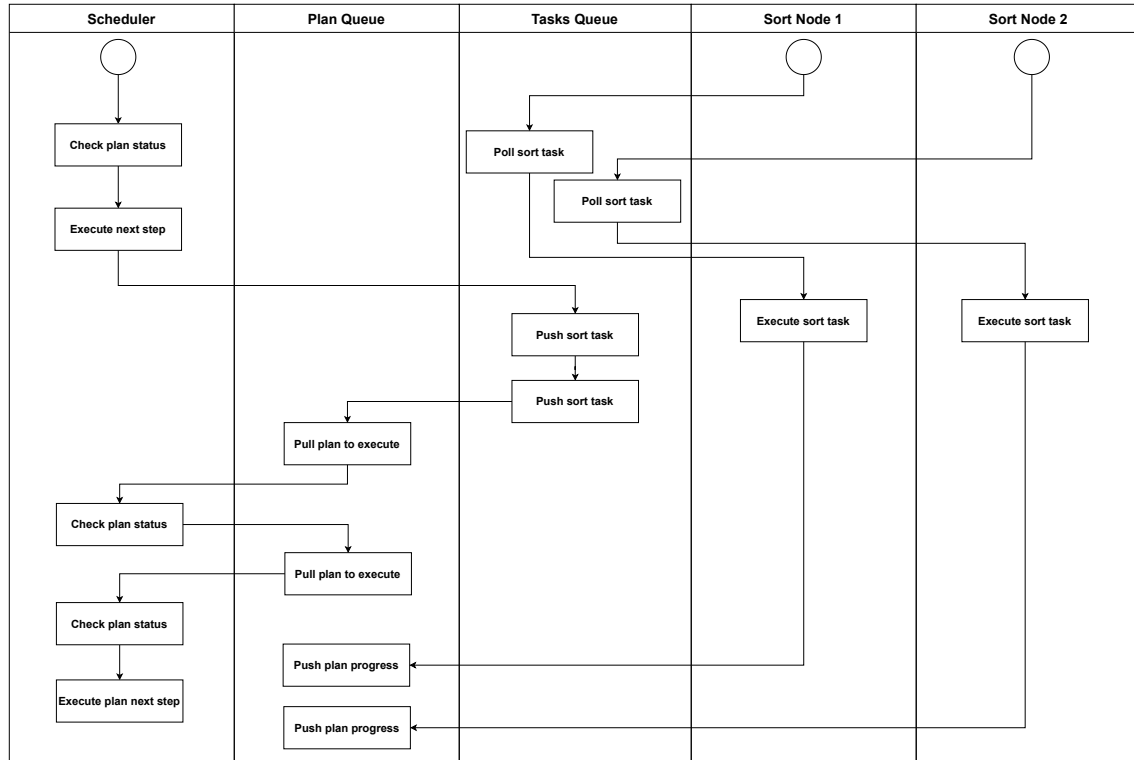


Figura 6: Diagrama de actividades

En cuanto a la conectividad general es interesante observar:

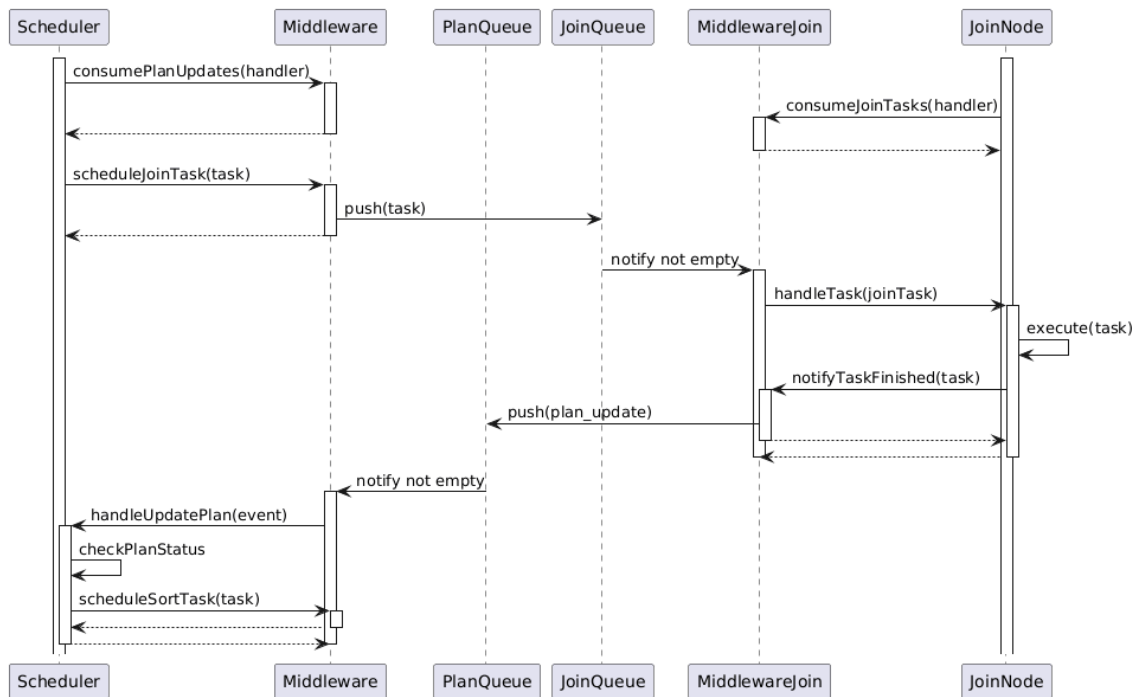


Figura 7: Diagrama de secuencia, como se conecta/consume eventos por medio de queues

Muestra como el middleware encapsula la conexión con las queues permitiendo cambiar entre, por ejemplo usar RabbitMQ o alguna otra implementación de Queues. Se observa que se mantiene la idea general de suscribirse a la queue y manejar mensajes por medio de un callback.

Entrando en algo más en detalle en partes importantes en el manejo de mensajes:

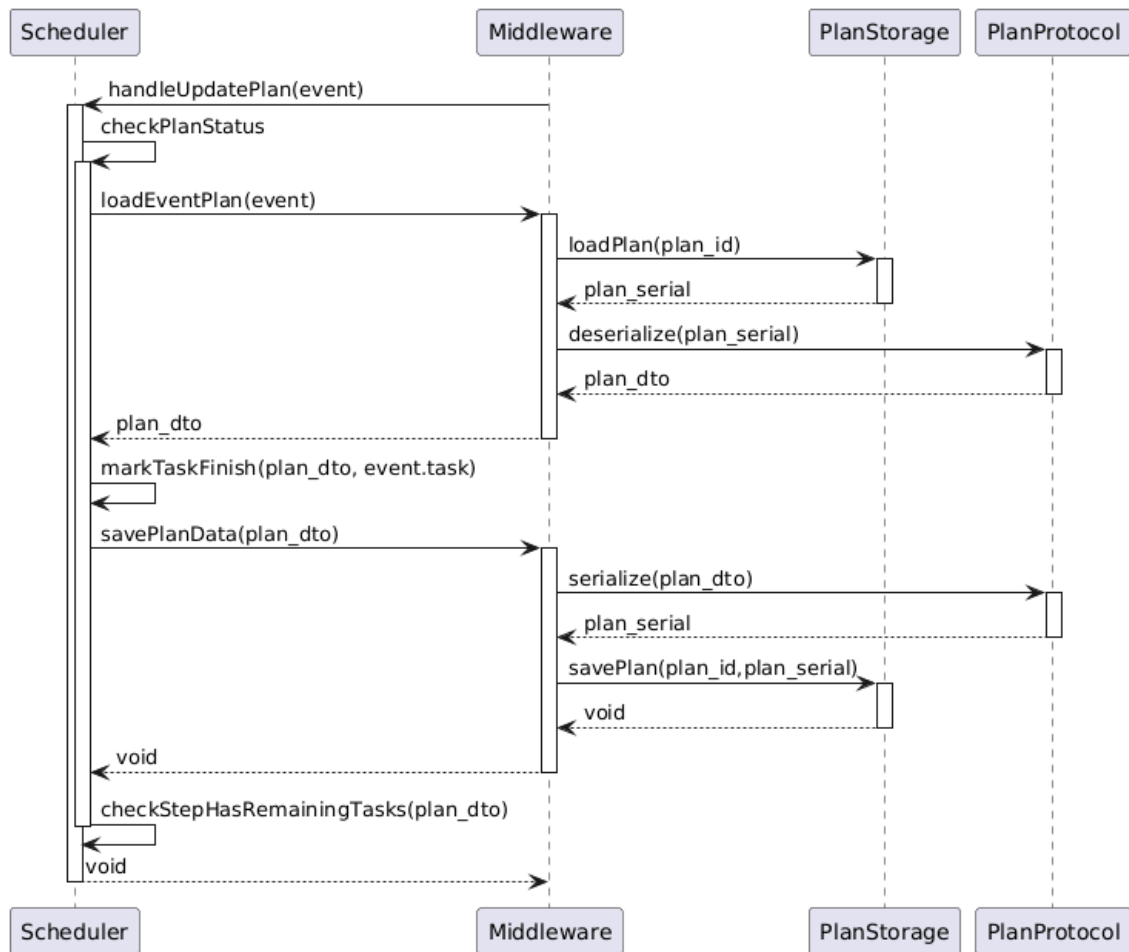


Figura 8: Diagrama de secuencia, en que consiste el chequeo del plan del scheduler y como se conecta con el storage cuando una tarea termina

El chequeo de estado de plan es lo primero que hace el scheduler cuando tiene que analizar que hacer despues de que algo ocurre con un plan, para esto por medio del Middleware obtiene el plan y en este caso que una tarea termina chequearia posteriormente si puede empezar el siguiente paso. Ya que en un paso del plan puede haber multiples tareas que se estan ejecutando paralelamente. Como podria ser multiples archivos input, como transacciones1 y transacciones2, o tambien podrian ser acciones en set de datos distintos, por ejemplo antes de un Join para los archivos de la izquierda o los de la derecha del join. Si se quisiese tener multiples schedulers apareceria la necesidad de que check plan status este protegida por un mutex, ya que al haber multiples tareas dentro de un mismo paso, y multiples schedulers que handlearian el update del plan cuando alguna termina. Puede ser que dos nodos schedulers distintos quieran acceder al mismo plan y modificar por ejemplo las tareas restantes del paso actual del plan.

Por otro lado:

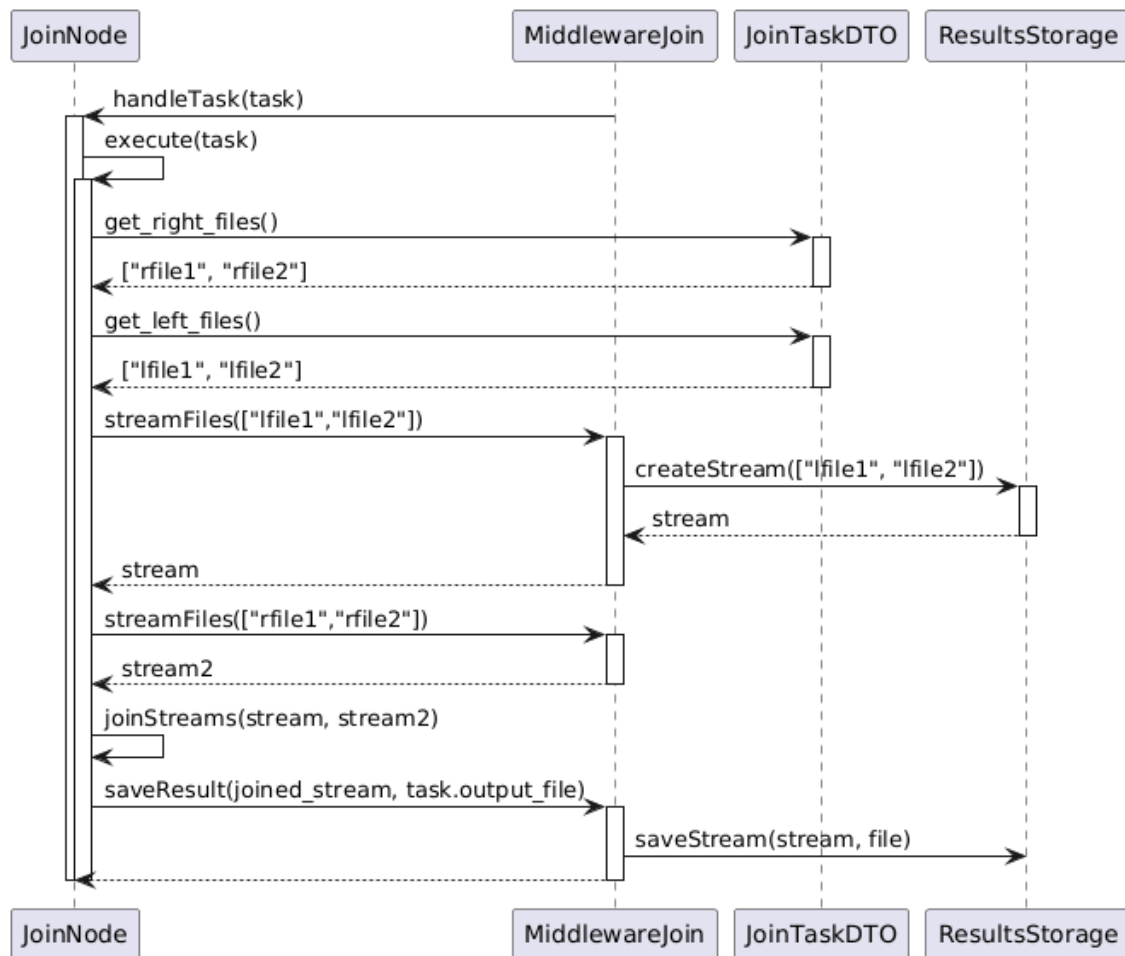


Figura 9: Diagrama de secuencia, como hace un nodo para leer archivos de input y escribir su output

Se observa la idea general de como un nodo lee el input del storage por medio del middleware y escribe su resultado en el storage. En el archivo especificado por la task, definido en el plan.

5. Vista de Desarrollo

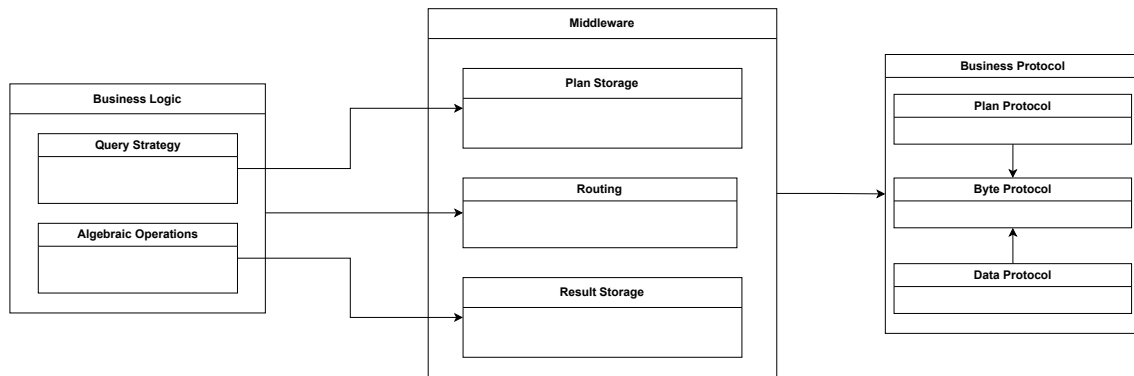


Figura 10: Diagrama de paquetes

6. Vista Física

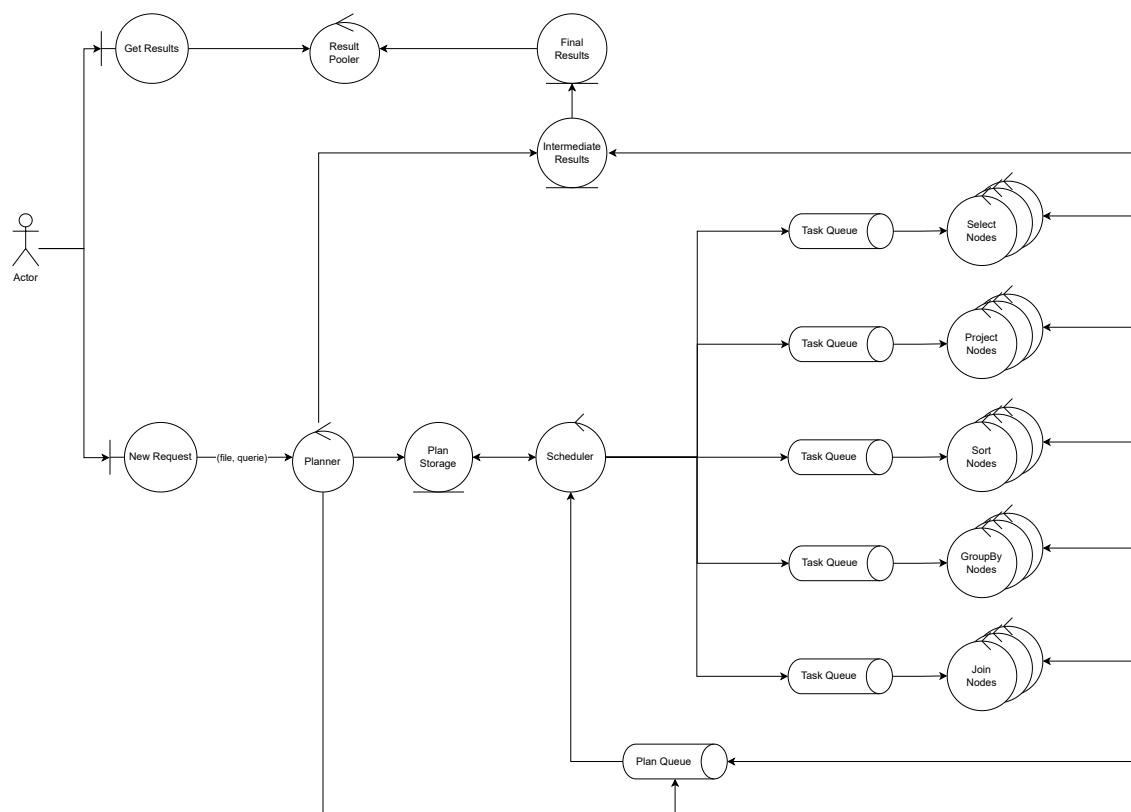


Figura 11: Diagrama de robustez

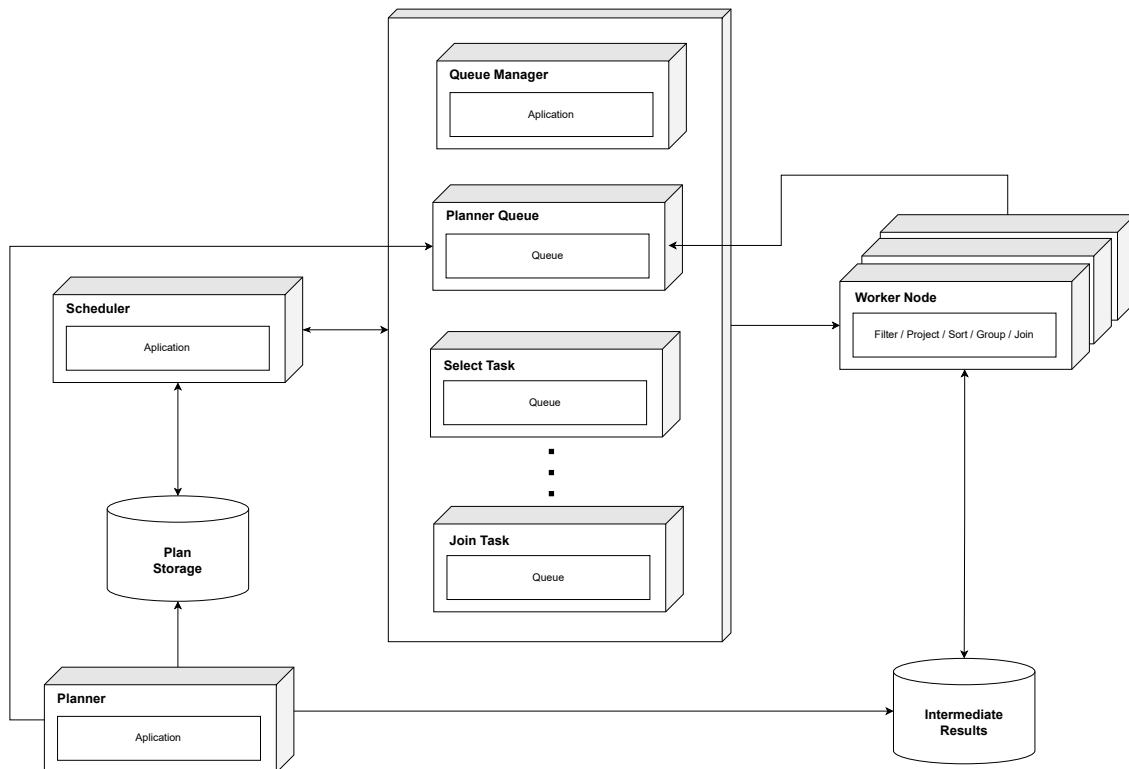


Figura 12: Diagrama de despliegue

7. Escenarios